HOMEWORK 7

Kavya Saranya K V
2025115152

1. Write a C program to construct an expression tree for a given expression. Eg: (a-b) + (c*d)

Code:

```
#include <stdio.h>
#include <stdlib.h>
struct Node {
    char data;
    struct Node *left, *right;
};
struct Node *stack[50];
int top = -1;
struct Node* newNode(char c) {
    struct Node *n = malloc(sizeof(struct Node));
    n->data = c;
    n->left = NULL;
    n->right = NULL;
    return n;
}
void inorder(struct Node *r) {
    if (r != NULL) {
        inorder(r->left);
        printf("%c ", r->data);
        inorder(r->right);
    }
}
void preorder(struct Node *r) {
    if (r != NULL) {
        printf("%c ", r->data);
        preorder(r->left);
        preorder(r->right);
    }
}
void postorder(struct Node *r) {
    if (r != NULL) {
        postorder(r->left);
        postorder(r->right);
        printf("%c ", r->data);
    }
```

```
}
int main() {
    char exp[] = "ab-cd*+";
    int i;
    for (i = 0; exp[i] != '\0'; i++) {
        struct Node *n = newNode(exp[i]);
        if (exp[i] == '+' || exp[i] == '-' || exp[i] == '*' ||
exp[i] == '/') {
            n->right = stack[top--];
            n->left = stack[top--];
        }
        stack[++top] = n;
    }
    struct Node *root = stack[top];
    printf("Postfix of (a-b)+(c*d): %s\n", exp);
    printf("Inorder:   ");
    inorder(root);
    printf("\nPreorder:  ");
    preorder(root);
    printf("\nPostorder: ");
    postorder(root);
    printf("\n");
    return 0;
}
```

Sample Output:

```
Postfix of (a-b)+(c*d): ab-cd*+
Inorder:   a - b + c * d
Preorder:  + - a b * c d
Postorder: a b - c d * +
```

2. Write a C program to check whether the given binary tree is a binary search tree.

Code:

```
#include <stdio.h>
#include <stdlib.h>
struct Node {
    int data;
    struct Node *left, *right;
};
int arr[50], n;
struct Node* newNode(int val) {
    struct Node *t = malloc(sizeof(struct Node));
    t->data = val;
    t->left = NULL;
```

```
    t->right = NULL;
    return t;
}
void inorder(struct Node *r) {
    if (r != NULL) {
        inorder(r->left);
        arr[n++] = r->data;
        inorder(r->right);
    }
}
int isBST(struct Node *root) {
    int i;
    n = 0;
    inorder(root);
    for (i = 1; i < n; i++)
        if (arr[i] <= arr[i - 1])
            return 0;
    return 1;
}
int main() {
    struct Node *t1 = newNode(50);
    t1->left = newNode(30);
    t1->right = newNode(70);
    t1->left->left = newNode(20);
    t1->left->right = newNode(40);
    t1->right->left = newNode(60);
    t1->right->right = newNode(80);
    struct Node *t2 = newNode(50);
    t2->left = newNode(30);
    t2->right = newNode(70);
    t2->left->left = newNode(20);
    t2->left->right = newNode(60);
    t2->right->left = newNode(65);
    t2->right->right = newNode(80);
    if (isBST(t1))
        printf("Tree 1 is a BST\n");
    else
        printf("Tree 1 is not a BST\n");
    if (isBST(t2))
        printf("Tree 2 is a BST\n");
    else
        printf("Tree 2 is not a BST\n");
    return 0;
}
```

Sample Output:

```
Tree 1 is a BST
Tree 2 is not a BST
```

3. Write a C program to implement a max heap.

Code:

```
#include <stdio.h>
int heap[100], size = 0;
void insert(int val) {
    int i = size, temp;
    heap[size++] = val;
    while (i > 0 && heap[(i - 1) / 2] < heap[i]) {
        temp = heap[i];
        heap[i] = heap[(i - 1) / 2];
        heap[(i - 1) / 2] = temp;
        i = (i - 1) / 2;
    }
}
int deleteMax() {
    int max = heap[0], i = 0, child, temp;
    heap[0] = heap[--size];
    while (2 * i + 1 < size) {
        child = 2 * i + 1;
        if (child + 1 < size && heap[child + 1] > heap[child])
            child++;
        if (heap[i] >= heap[child])
            break;
        temp = heap[i];
        heap[i] = heap[child];
        heap[child] = temp;
        i = child;
    }
    return max;
}
void display() {
    int i;
    for (i = 0; i < size; i++)
        printf("%d ", heap[i]);
    printf("\n");
}
int main() {
    int vals[] = {20, 15, 30, 10, 40, 25, 35};
    int i;
    for (i = 0; i < 7; i++)
        insert(vals[i]);
```

```
    printf("Max heap: ");
    display();
    printf("Deleted max: %d\n", deleteMax());
    printf("Heap now: ");
    display();
    printf("Deleted max: %d\n", deleteMax());
    printf("Heap now: ");
    display();
    return 0;
}
```

Sample Output:

```
Max heap: 40 30 35 10 15 20 25
Deleted max: 40
Heap now: 35 30 25 10 15 20
Deleted max: 35
Heap now: 30 20 25 10 15
```

4. Given an undirected graph, the task is to check if there is a cycle in the given graph.

Code:

```
#include <stdio.h>
#define N 6
int g[N][N], visited[N];
int dfs(int v, int parent) {
    int u;
    visited[v] = 1;
    for (u = 0; u < N; u++) {
        if (g[v][u] == 1) {
            if (visited[u] == 0) {
                if (dfs(u, v) == 1)
                    return 1;
            } else if (u != parent)
                return 1;
        }
    }
    return 0;
}
int hasCycle() {
    int i;
    for (i = 0; i < N; i++)
        visited[i] = 0;
    for (i = 0; i < N; i++)
        if (visited[i] == 0 && dfs(i, -1) == 1)
            return 1;
```

```
    return 0;
}
void addEdge(int a, int b) {
    g[a][b] = 1;
    g[b][a] = 1;
}
int main() {
    addEdge(0, 1);
    addEdge(1, 2);
    addEdge(2, 3);
    addEdge(4, 5);
    printf("Edges: 0-1 1-2 2-3 4-5\n");
    if (hasCycle())
        printf("Cycle found\n");
    else
        printf("No cycle\n");
    addEdge(3, 1);
    printf("Added edge 3-1\n");
    if (hasCycle())
        printf("Cycle found\n");
    else
        printf("No cycle\n");
    return 0;
}
```

Sample Output:

```
Edges: 0-1 1-2 2-3 4-5
No cycle
Added edge 3-1
Cycle found
```

5. Given an n x m grid of 'W' (water) and 'L' (land), the task is to count the number of islands. Input: grid[][] = {{L, L, W, W, W}, {W, L, W, W, L}, {L, W, W, L, L}, {W, W, W, W, W}, {L, W, L, L, W}}. Output: 4 islands.

Code:

```
#include <stdio.h>
#define R 5
#define C 5
char grid[R][C] = {
    {'L', 'L', 'W', 'W', 'W'},
    {'W', 'L', 'W', 'W', 'L'},
    {'L', 'W', 'W', 'L', 'L'},
    {'W', 'W', 'W', 'W', 'W'},
    {'L', 'W', 'L', 'L', 'W'}
};
```

```
int visited[R][C];
void dfs(int r, int c) {
    int i, j;
    if (r < 0 || r >= R || c < 0 || c >= C)
        return;
    if (grid[r][c] == 'W' || visited[r][c] == 1)
        return;
    visited[r][c] = 1;
    for (i = -1; i <= 1; i++)
        for (j = -1; j <= 1; j++)
            dfs(r + i, c + j);
}
int main() {
    int i, j, count = 0;
    for (i = 0; i < R; i++)
        for (j = 0; j < C; j++)
            if (grid[i][j] == 'L' && visited[i][j] == 0) {
                dfs(i, j);
                count++;
            }
    printf("Number of islands = %d\n", count);
    return 0;
}
```

Sample Output:

```
Number of islands = 4
```

6. Find out the minimum spanning tree using Kruskal's algorithm.

Code:

```
#include <stdio.h>
int parent[10];
int find(int x) {
    while (parent[x] != x)
        x = parent[x];
    return x;
}
int main() {
    int n = 6, m = 7;
    int u[] = {0, 0, 1, 1, 2, 3, 4};
    int v[] = {1, 2, 2, 3, 3, 4, 5};
    int w[] = {4, 3, 1, 2, 4, 2, 6};
    int i, j, t, total = 0;
    for (i = 0; i < m - 1; i++) {
        for (j = 0; j < m - 1 - i; j++) {
            if (w[j] > w[j + 1]) {
```

```
                t = w[j]; w[j] = w[j + 1]; w[j + 1] = t;
                t = u[j]; u[j] = u[j + 1]; u[j + 1] = t;
                t = v[j]; v[j] = v[j + 1]; v[j + 1] = t;
            }
        }
    }
    for (i = 0; i < n; i++)
        parent[i] = i;
    printf("Edges in the MST:\n");
    for (i = 0; i < m; i++) {
        int a = find(u[i]);
        int b = find(v[i]);
        if (a != b) {
            parent[a] = b;
            printf("%d - %d (weight %d)\n", u[i], v[i], w[i]);
            total = total + w[i];
        }
    }
    printf("Total weight = %d\n", total);
    return 0;
}
```

Sample Output:

```
Edges in the MST:
1 - 2 (weight 1)
1 - 3 (weight 2)
3 - 4 (weight 2)
0 - 2 (weight 3)
4 - 5 (weight 6)
Total weight = 14
```

7. Use Dijkstra's algorithm to find shortest path from a source to all nodes.

Code:

```
#include <stdio.h>
#define N 6
#define INF 9999
int main() {
    int g[N][N] = {
        {0, 7, 9, 0, 0, 14},
        {7, 0, 10, 15, 0, 0},
        {9, 10, 0, 11, 0, 2},
        {0, 15, 11, 0, 6, 0},
        {0, 0, 0, 6, 0, 9},
        {14, 0, 2, 0, 9, 0}
    };
```

```
    int dist[N], done[N];
    int src = 0, i, j, u, min;
    for (i = 0; i < N; i++) {
        dist[i] = INF;
        done[i] = 0;
    }
    dist[src] = 0;
    for (i = 0; i < N; i++) {
        min = INF;
        for (j = 0; j < N; j++)
            if (done[j] == 0 && dist[j] < min) {
                min = dist[j];
                u = j;
            }
        done[u] = 1;
        for (j = 0; j < N; j++)
            if (g[u][j] != 0 && dist[u] + g[u][j] < dist[j])
                dist[j] = dist[u] + g[u][j];
    }
    printf("Shortest distance from source %d:\n", src);
    for (i = 0; i < N; i++)
        printf("To %d = %d\n", i, dist[i]);
    return 0;
}
```

Sample Output:

```
Shortest distance from source 0:
To 0 = 0
To 1 = 7
To 2 = 9
To 3 = 20
To 4 = 20
To 5 = 11
```

8. Write a program to find a mother vertex in the graph. A mother vertex in a graph G = (V, E) is a vertex v such that all other vertices in G can be reached by a path from v.

Code:

```
#include <stdio.h>
#define N 7
int g[N][N], visited[N];
void dfs(int v) {
    int u;
    visited[v] = 1;
    for (u = 0; u < N; u++)
```

```
        if (g[v][u] == 1 && visited[u] == 0)
            dfs(u);
}
int main() {
    int a[] = {0, 0, 1, 4, 5, 5, 6, 6};
    int b[] = {1, 2, 3, 1, 2, 6, 0, 4};
    int i, j, count, found = 0;
    for (i = 0; i < 8; i++)
        g[a[i]][b[i]] = 1;
    for (i = 0; i < N; i++) {
        for (j = 0; j < N; j++)
            visited[j] = 0;
        dfs(i);
        count = 0;
        for (j = 0; j < N; j++)
            count = count + visited[j];
        if (count == N) {
            printf("Mother vertex = %d\n", i);
            found = 1;
        }
    }
    if (found == 0)
        printf("No mother vertex\n");
    return 0;
}
```

Sample Output:

```
Mother vertex = 5
```

9. A directed graph is strongly connected if there is a path between all pairs of vertices. A strongly connected component (SCC) of a directed graph is a maximal strongly connected subgraph. Given a graph, print all SCC.

Code:

```
#include <stdio.h>
#define N 5
int main() {
    int reach[N][N] = {0};
    int a[] = {1, 0, 2, 0, 3};
    int b[] = {0, 2, 1, 3, 4};
    int i, j, k;
    for (i = 0; i < 5; i++)
        reach[a[i]][b[i]] = 1;
    for (i = 0; i < N; i++)
        reach[i][i] = 1;
    for (k = 0; k < N; k++)
```

```
        for (i = 0; i < N; i++)
            for (j = 0; j < N; j++)
                if (reach[i][k] == 1 && reach[k][j] == 1)
                    reach[i][j] = 1;
    int printed[N] = {0};
    int count = 0;
    for (i = 0; i < N; i++) {
        if (printed[i] == 0) {
            count++;
            printf("SCC %d: ", count);
            for (j = 0; j < N; j++)
                if (reach[i][j] == 1 && reach[j][i] == 1) {
                    printf("%d ", j);
                    printed[j] = 1;
                }
            printf("\n");
        }
    }
    return 0;
}
```

Sample Output:

```
SCC 1: 0 1 2
SCC 2: 3
SCC 3: 4
```

10. A bipartite graph G = (V, E) is a graph such that V can be partitioned into two subsets V1 and V2 and no edge has both its vertices in the same subset. Write a program to determine whether a graph is bipartite graph.

Code:

```
#include <stdio.h>
#define N 6
int g[N][N], color[N];
int dfs(int v, int c) {
    int u;
    color[v] = c;
    for (u = 0; u < N; u++) {
        if (g[v][u] == 1) {
            if (color[u] == -1) {
                if (dfs(u, 1 - c) == 0)
                    return 0;
            } else if (color[u] == c)
                return 0;
        }
    }
```

```
    return 1;
}
int isBipartite() {
    int i;
    for (i = 0; i < N; i++)
        color[i] = -1;
    for (i = 0; i < N; i++)
        if (color[i] == -1 && dfs(i, 0) == 0)
            return 0;
    return 1;
}
void addEdge(int a, int b) {
    g[a][b] = 1;
    g[b][a] = 1;
}
int main() {
    addEdge(0, 1);
    addEdge(1, 2);
    addEdge(2, 3);
    addEdge(3, 0);
    addEdge(4, 5);
    printf("Edges: 0-1 1-2 2-3 3-0 4-5\n");
    if (isBipartite())
        printf("Bipartite graph\n");
    else
        printf("Not a bipartite graph\n");
    addEdge(0, 2);
    printf("Added edge 0-2\n");
    if (isBipartite())
        printf("Bipartite graph\n");
    else
        printf("Not a bipartite graph\n");
    return 0;
}
```

Sample Output:

```
Edges: 0-1 1-2 2-3 3-0 4-5
Bipartite graph
Added edge 0-2
Not a bipartite graph
```